

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper

data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. (5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters. (5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. (5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. (5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. (5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. (5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or

imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. (5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Perceptron Model Fails to Converge

Possible Causes

- The learning rate may be too high, causing the weights to oscillate, or too low, resulting in very slow learning.
- The initial weights may not be chosen properly.
- The bias term may be missing or updated incorrectly.
- The perceptron learning rule may contain implementation errors.
- The dataset may not actually be linearly separable.

Debugging Steps

- Verify that the dataset is linearly separable.
- Check whether the weights and bias are updated correctly after every iteration.
- Monitor the classification error for each epoch.
- Test the algorithm on a small sample dataset with known outputs.

Optimization Techniques

- Use a suitable learning rate.
- Initialize weights with small random values.

- Shuffle the training data before each epoch.
- Include and update the bias correctly to achieve stable convergence.

2. Multilayer Perceptron Shows Slow Convergence

Possible Causes

- Vanishing gradients prevent effective weight updates.
- Sigmoid or tanh activation functions may slow learning.
- Poor weight initialization affects gradient flow.
- An unsuitable learning rate increases training time.
- The optimizer may not be efficient.

Debugging Steps

- Observe the training and validation loss curves.
- Check whether gradients become too small.
- Experiment with different activation functions.
- Compare the performance with different learning rates.

Optimization Techniques

- Replace sigmoid with ReLU activation.
- Use Xavier or He weight initialization.
- Normalize input features.
- Use Adam optimizer with Batch Normalization.
- Apply early stopping to avoid unnecessary training.

3. Genetic Algorithm Produces Suboptimal Solutions

Possible Causes

- Chromosomes may not represent the solution correctly.
- The fitness function may not accurately evaluate solutions.
- Premature convergence reduces population diversity.
- Mutation rate may be too low.

Debugging Steps

- Validate chromosome encoding.
- Verify the correctness of the fitness function.
- Monitor diversity across generations.
- Plot the best fitness value over iterations.

Optimization Techniques

- Increase mutation probability.

- Use elitism to preserve the best solutions.
- Adjust crossover and selection methods.
- Increase the population size to improve diversity.

4. Deep Neural Network with Genetic Programming Overfits

Possible Causes

- The network has too many parameters.
- Training data is insufficient.
- The model memorizes the training data.
- Genetic programming increases computational complexity.

Debugging Steps

- Compare training and validation accuracy.
- Check whether validation loss increases while training loss decreases.
- Analyze model complexity.
- Monitor execution time.

Optimization Techniques

- Apply L1/L2 regularization.
- Use Dropout layers.
- Prune unnecessary neurons.
- Use early stopping.
- Simplify the network architecture to improve generalization.

5. Decision Tree Shows Poor Generalization

Possible Causes

- The decision tree is too deep.
- The training data contains noisy samples.
- Too many irrelevant features are used.
- The model memorizes training examples.

Debugging Steps

- Compare training and testing accuracy.
- Visualize the tree depth.
- Identify noisy or duplicate records.
- Check feature importance.

Optimization Techniques

- Apply pre-pruning or post-pruning.

- Perform cross-validation.
- Select only important features.
- Limit the maximum tree depth.

6. CNN Produces Unstable Results

Possible Causes

- Improper data augmentation.
- Vanishing or exploding gradients.
- Incorrect Batch Normalization settings.
- High learning rate.

Debugging Steps

- Monitor accuracy and loss during training.
- Verify augmented images.
- Inspect gradient values.
- Compare different batch sizes.

Optimization Techniques

- Use Batch Normalization.
- Apply learning rate scheduling.
- Use appropriate image augmentation.
- Adopt Adam optimizer.
- Reduce the learning rate for stable training.

7. Reinforcement Learning Agent Fails

Possible Causes

- Poorly designed reward function.
- Limited exploration of the environment.
- Incorrect state representation.
- Inappropriate hyperparameters.

Debugging Steps

- Observe cumulative rewards.
- Analyze the agent's actions.
- Verify state transitions.
- Compare exploration and exploitation rates.

Optimization Techniques

- Use reward shaping.

- Apply ϵ -greedy exploration.
- Tune learning rate and discount factor.
- Improve state representation.

8. K-Means Produces Inconsistent Clusters

Possible Causes

- Random centroid initialization.
- Incorrect number of clusters.
- Outliers influence centroids.
- Features have different scales.

Debugging Steps

- Visualize the clusters.
- Monitor centroid movement.
- Calculate the silhouette score.
- Detect outliers.

Optimization Techniques

- Use K-Means++ initialization.
- Normalize the dataset.
- Select K using the Elbow method.
- Remove outliers before clustering.

9. SVM Underperforms on Nonlinear Data

Possible Causes

- Incorrect kernel selection.
- Improper values of C and gamma.
- Features are not scaled.
- Presence of noisy data.

Debugging Steps

- Compare Linear, Polynomial, and RBF kernels.
- Scale the input features.
- Evaluate confusion matrix and accuracy.
- Analyze support vectors.

Optimization Techniques

- Use the RBF kernel.
- Normalize features.

- Perform Grid Search for C and gamma.
- Remove irrelevant features.

10. RNN Cannot Learn Long-Term Dependencies

Possible Causes

- Vanishing gradients.
- Limited memory capacity.
- Long input sequences.
- Inefficient architecture.

Debugging Steps

- Monitor gradient values.
- Compare predicted and actual outputs.
- Analyze hidden state behavior.
- Observe training loss.

Optimization Techniques

- Replace RNN with LSTM or GRU.
- Apply gradient clipping.
- Use attention mechanisms.
- Normalize sequential data.

11. SVM Shows Low Accuracy

Possible Causes

- Wrong kernel selection.
- Poor hyperparameter values.
- Lack of feature scaling.
- Noisy or irrelevant features.

Debugging Steps

- Test different kernels.
- Scale the features.
- Analyze confusion matrix.
- Validate support vectors.

Optimization Techniques

- Perform Grid Search.
- Normalize the dataset.
- Tune C and gamma.

- Remove unnecessary features.

12. K-Means Creates Imbalanced Clusters

Possible Causes

- Incorrect K value.
- Poor centroid initialization.
- Outliers.
- Uneven data distribution.

Debugging Steps

- Visualize clusters.
- Check centroid positions.
- Evaluate silhouette score.
- Detect outliers.

Optimization Techniques

- Apply K-Means++.
- Use the Elbow method.
- Normalize data.
- Remove outliers before clustering.

13. RNN Gives Inconsistent Time-Series Predictions

Possible Causes

- Vanishing gradients.
- Short sequence length.
- Poor architecture.
- Improper learning rate.

Debugging Steps

- Monitor training and validation loss.
- Compare predictions with actual values.
- Inspect gradients.
- Verify sequence length.

Optimization Techniques

- Use LSTM/GRU.
- Apply gradient clipping.
- Normalize sequences.
- Tune optimizer and learning rate.

14. Naïve Bayes Performs Poorly

Possible Causes

- Feature independence assumption is violated.
- Presence of noisy and irrelevant words.
- Class imbalance.
- Poor text preprocessing.

Debugging Steps

- Analyze important features.
- Remove stop words and punctuation.
- Check class distribution.
- Evaluate confusion matrix.

Optimization Techniques

- Apply feature selection.
- Use Laplace smoothing.
- Balance the dataset.
- Perform effective preprocessing using TF-IDF.

15. Reinforcement Learning Navigation Agent Fails

Possible Causes

- Sparse reward signals.
- Poor exploration strategy.
- Incorrect reward function.
- Improper hyperparameters.

Debugging Steps

- Monitor cumulative rewards.
- Observe the learned policy.
- Analyze state transitions.
- Evaluate exploration rate.

Optimization Techniques

- Apply reward shaping.
- Tune epsilon decay.
- Use policy optimization algorithms such as PPO.
- Optimize learning rate and discount factor.